System & Architecture Design Document (SDD)

Etel Ride Platform

Version 1.1 | Etel Group

Prepared for the INSA Cyber-Security Audit application (Etel Ride scope), submitted in support of the Road Transport license application with the Ministry of Transport and Logistics.

Document contents

1. Architectural Overview
2. Technology Stack
3. Key Components
4. System Architecture Diagram
5. Data Flow Diagrams (Level 0 and Level 1)
6. Data Flow Explanation
7. Module Design
8. Data Architecture
9. Interface Design
10. Deployment Architecture
11. Traceability

> Version 1.1 adds Sections 2 to 6 (Technology Stack, Key Components, System Architecture Diagram, Data Flow Diagram and Data Flow Explanation) requested by the INSA reviewer, and is supplied as PDF. All content is derived from the production source under /delivery/ in the Etel-FTBG repository.

1. Architectural Overview

Etel Ride follows a single-entrypoint, controller-dispatch architecture shared with Etel Delivery: every client request (from the Ride or Driver app, or from server-to-server integrations) is routed through one file, api/boot.php, which authenticates the caller (bearer token or shared service key), then dispatches by an 'action' name to the relevant controller under api/controllers/. There is no separate framework/router layer, and no separate deployment for Ride versus Delivery — they are one backend serving two service lines.

1.1 Scope

Etel Ride shares its backend and database with Etel Delivery, Etel Group's parcel and food delivery service line; both are served by the same api/boot.php dispatch entrypoint. This SDD documents the architecture as it relates to Ride. Shared components (authentication, the admin access-control mechanism, hosting) are described because Ride depends on them. Delivery-only modules are noted as out of scope rather than omitted silently.

1.2 Explicitly out of scope

The following components exist in the same codebase/repository but serve Etel Delivery, not Etel Ride, and are not described further in this document set:

- OrderController.php — delivery order lifecycle
- ServiceGiverController.php — storefront/business operations
- Courier and Service Giver roles and their apps (courier/, servicegiver.html)
- Delivery-zone, delivery-tier, and item-catalog data and admin tooling
- DocumentsController.php, SystemResetController.php, EquityController.php, PlatformEconomicsController.php, PlanningController.php — platform-wide admin tooling unrelated to the ride-hailing function specifically (internal document library, full-system data reset, investor/equity management, cost-planning tools)

2. Technology Stack

Layer	Technology	Where it is used
Client	HTML5, CSS3, vanilla JavaScript (no framework); Progressive Web App; installable as Android Trusted Web Activity (TWA)	ride/, driver/, admin.html, login.html; per-app manifest files
Client build	Node.js + terser: per-app source compiled to minified bundles, synced to versioned release folders (stable/canary channels)	js/build-bundles.js, js/releases/{app}/{version}/
Offline / caching	Service worker: cache-first static assets, network-first API calls with 5-minute cache, offline fallback page	sw.js, offline.html
Maps and location	Google Maps JavaScript API (map, markers, polylines, Places Autocomplete), Geocoding API, browser Geolocation	ride and driver apps; server-side geocoding helpers
Server language	PHP 8.1 (procedural, controller-dispatch); no third-party framework	api/boot.php, api/controllers/*
Database	MySQL / MariaDB, utf8mb4, accessed through PDO with prepared statements (emulated prepares disabled); session time zone aligned with PHP (Africa/Addis_Ababa)	api/boot.php, api/migrations/*.sql
Authentication	Custom HMAC-SHA256 signed bearer token with token_version revocation; EtelID SSO; shared service key (hash_equals) for machine-to-machine calls	boot.php, AuthController, etelid_auth.php
Real-time	Server-Sent Events (SSE) for live ride status and driver location	sse_ride_status, sse_driver_stream
Notifications and alerting	Web Push with VAPID (RFC 8292); Telegram Bot API for operational alerts	api/webpush.php, push.php, telegram_webhook.php
Payments integration	EtelPay server-to-server transaction verification (etel_pay_verify_tx)	api/etel_pay_bridge.php
Background jobs	cPanel cron calling key-protected PHP scripts	cron_close_stuck_rides.php, cron_dispatch_scheduled.php, equb_cron.php
Hosting and deployment	Shared cPanel hosting (Apache/LiteSpeed), HTTPS; no SSH; deploys through in-app Admin Updater with backup and patch_log; GitHub Actions workflow for zip-on-merge	api/updater.php, .github/workflows/deploy-zip-on-merge.yml

3. Key Components

Component	Type	Responsibility	Trust boundary / security role
Ride app (passenger)	Client PWA/TWA	Registration and login, fare estimate, booking, live tracking, chat, SOS, ratings, favourite and scheduled rides	Untrusted. Holds bearer token only; every action re-authorised server-side.

Component	Type	Responsibility	Trust boundary / security role
Driver app	Client PWA/TWA	Onboarding, go online/offline, GPS pings, accept/arrive/start/complete, earnings, cash-out, top-up	Untrusted. Privileged actions gated by rides_require_onboarded().
Admin console	Client web app	Pricing, driver approvals, vehicle-change review, issue and SOS handling, release channels	Untrusted client; each action requires admin role plus capability via role_permissions.
boot.php	Gateway	Request parsing, token / service-key authentication, rate limiting, action whitelist, dispatch	Single choke point for all API traffic.
Controllers	Business logic	Auth, Ride, Admin, Equb, Agent, Nfc, DriverFuel, Push	Ownership checks per ride; capability checks per admin action.
ride_fare.php / hexgrid.php	Library	Fare calculation shared by the app and EtelAI; hex-grid nearest-driver matching	Pure computation; no external input trusted.
SSE endpoints	Real-time	Push ride status and driver position to the participants	Authenticated; scoped to the caller's own ride.
upload.php	Service	Driver selfie, licence and document uploads	Content-sniffed MIME whitelist, size cap, server-generated file names.
Cron scripts	Background	Close stuck rides, dispatch scheduled rides, Equb processing	Standalone, protected by a secret key from .env.
MySQL database	Data store	Users, rides, drivers, GPS path, fares, payouts, permissions	Reached only through PDO prepared statements.
EtelID, EtelPay, EtelAI	External group systems	Identity / SSO, wallet verification, AI booking agent	Shared service key over HTTPS, constant-time comparison.
Google Maps, Telegram, Web Push	External third parties	Maps and geocoding, operational alerts, browser notifications	API key / bot token / VAPID key held server- or domain-restricted.

3.1 Component inventory (files)

Component	Responsibility
api/boot.php	Shared dispatch entrypoint: request parsing, auth (token or service key), rate limiting, action routing. Serves both Ride and Delivery.
api/controllers/RideController.php	Ride-hailing lifecycle: fare estimation, booking, driver matching, ride state machine, driver earnings/compliance
api/controllers/AuthController.php	Identity & session (shared with Delivery)
api/controllers/AdminController.php	Platform administration; only its ride fare/pricing, driver-account, and ride/driver-issue-resolution actions are in scope here. Its order/service-giver/zone management actions serve Delivery only and are out of scope for this document.
api/controllers/EqubController.php	Rotating-savings benefit, used by drivers (and, out of scope here, couriers)
api/controllers/AgentController.php	EtelAI machine-to-machine integration; only its ride_* actions are in scope

Component	Responsibility
api/controllers/NfcController.php	EtelID NFC-tap bridge; only nfc_start_ride is in scope (its courier-pickup/delivery actions serve Delivery)
api/lib/ride_fare.php	Ride fare calculation, shared identically between the Ride app and the EtelAI agent integration
api/lib/hexgrid.php	Geospatial hex-grid driver matching
Ride app / Driver app	The two Ride-facing PWAs, each independently versioned and installable as an Android TWA
sw.js (service worker)	Offline caching, install/activate lifecycle, cache versioning (shared mechanism, per-app cache namespace)
EtelID (external)	Group-wide identity provider; SSO login, driver identity cross-check, NFC-tap bridge
EtelPay (external)	Wallet/payment app; verifies driver Etel Money top-up transactions

4. System Architecture Diagram

Etel Ride is a four-tier system. Thin PWA/TWA clients talk over HTTPS to a single PHP gateway (boot.php), which authenticates every request and dispatches it to a controller. Controllers read and write the MySQL database and call a small set of external services. Nothing in the client tier is trusted: authorization, ownership and fare calculation are all enforced on the server.

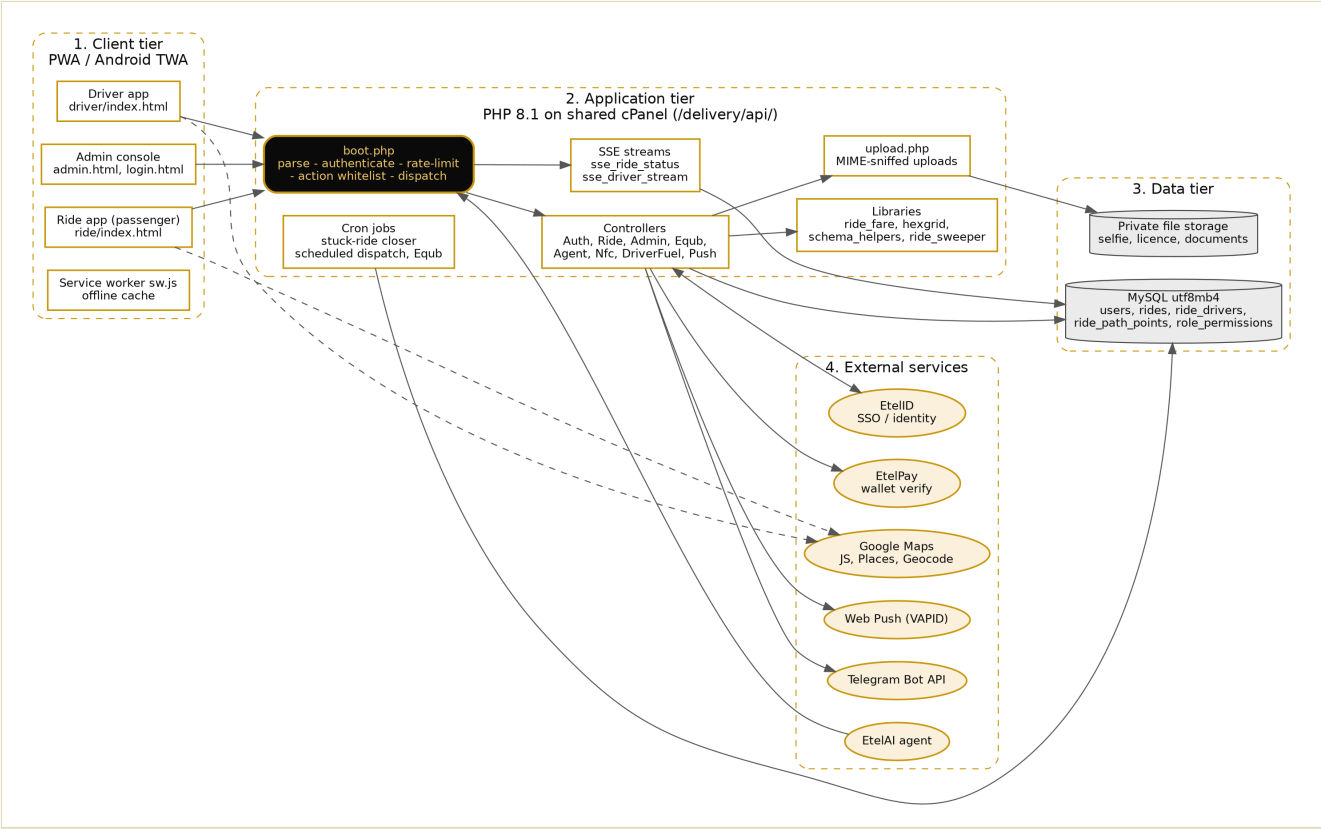


Figure 1 - Etel Ride system architecture (dashed arrows: browser-direct calls to Google Maps)

5. Data Flow Diagrams

5.1 Level 0 - context diagram

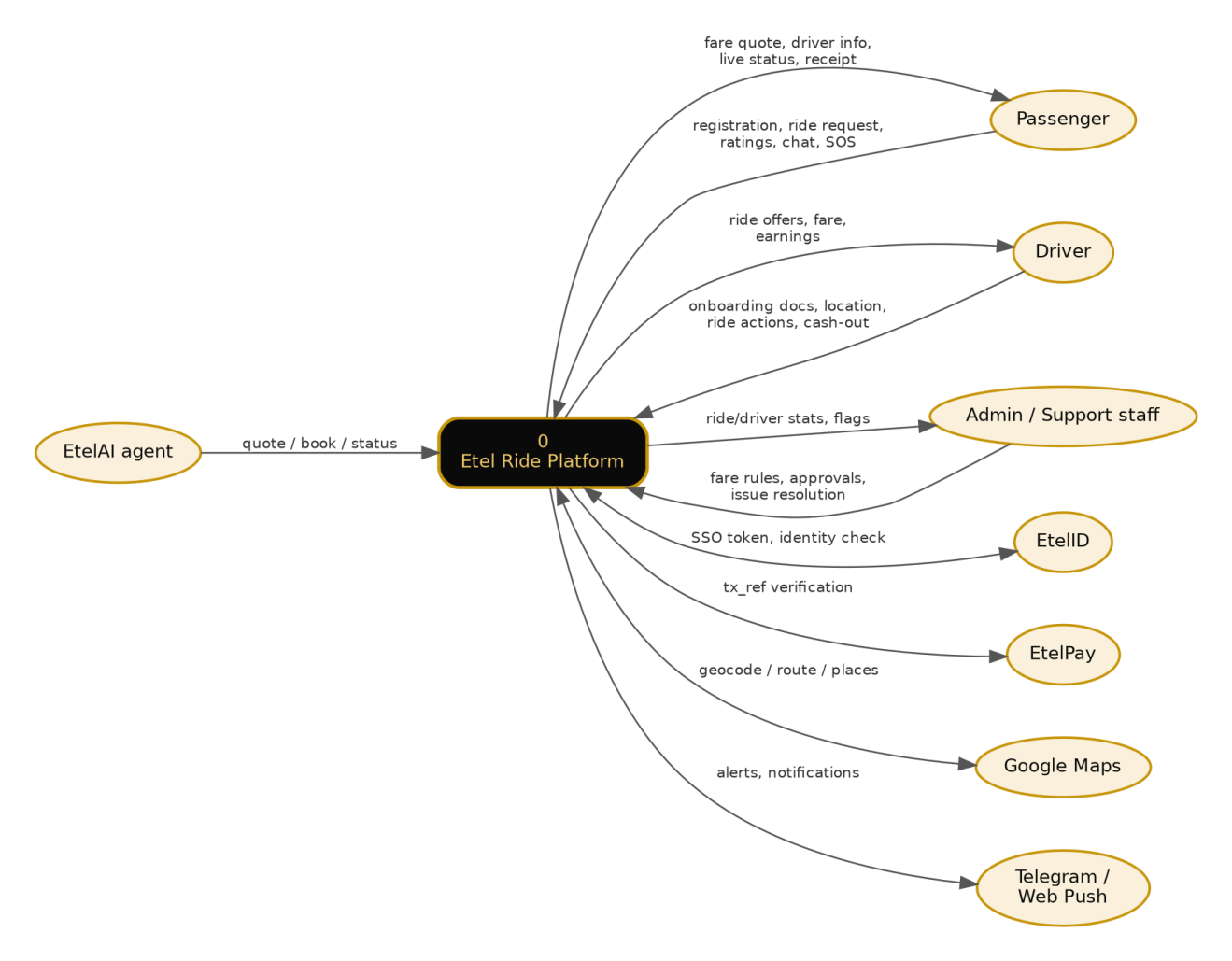


Figure 2 - Level 0 DFD: Etel Ride and its external entities

5.2 Level 1 - process decomposition

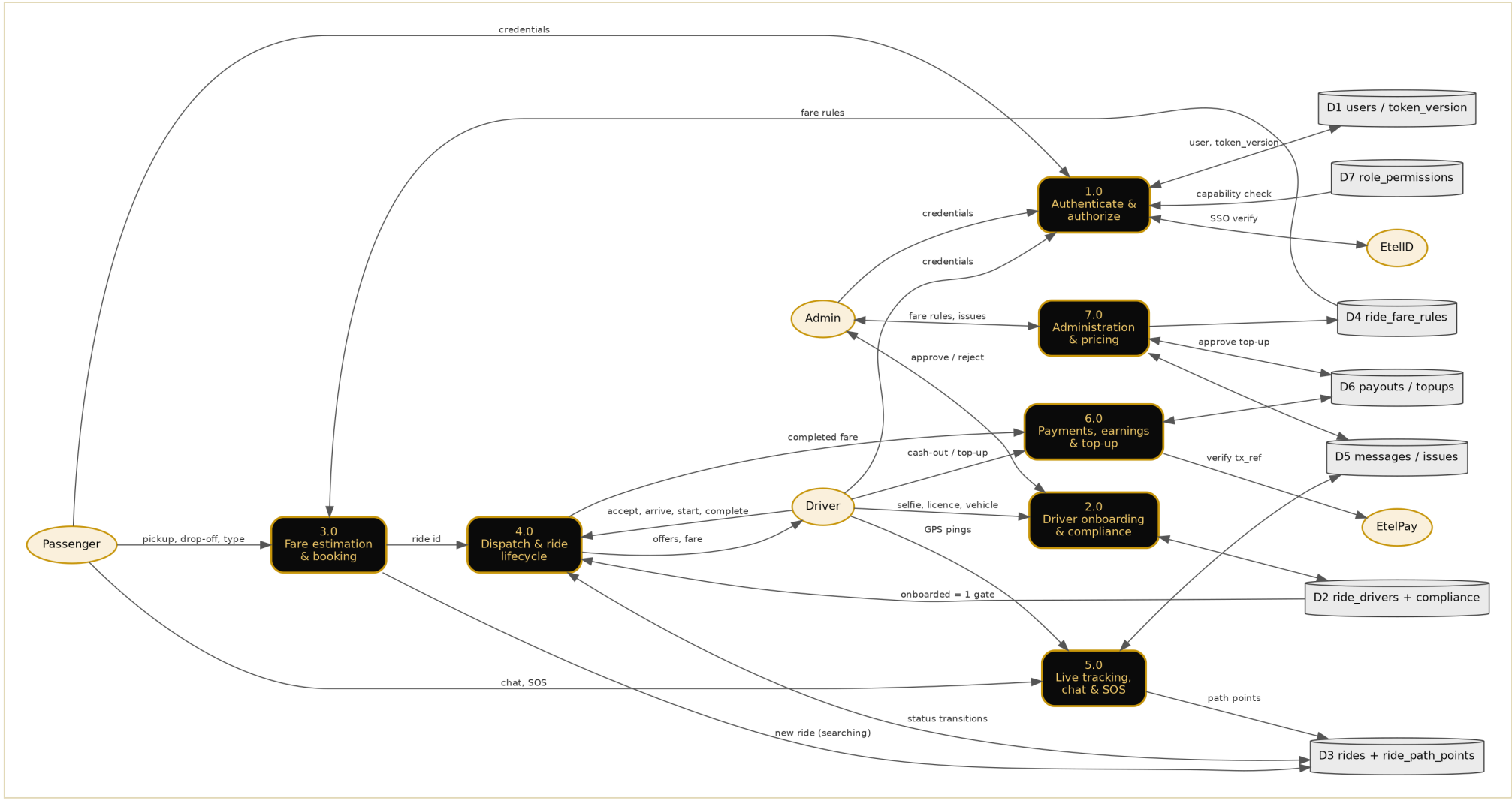


Figure 3 - Level 1 DFD: processes 1.0 to 7.0, data stores D1 to D7, external entities

6. Data Flow Explanation

6.1 External entities

Entity	Sends to Etel Ride	Receives from Etel Ride
Passenger	Registration, login, pickup and drop-off, vehicle type, chat, SOS, ratings	Fare quote, nearby drivers, driver details, live status and location, receipt
Driver	Onboarding documents (selfie, licence, vehicle), GPS pings, ride actions, cash-out and top-up requests	Ride offers, fare, earnings, announcements
Admin / support staff	Fare rules, driver and vehicle-change decisions, issue resolution, top-up approval	Ride and driver statistics, flagged identities, SOS and issue queues
EtelID	SSO token verification result, role check, NFC-tap ride start	SSO token, driver identity cross-check request
EtelPay	Transaction verification result	tx_ref, expected sender and amount
Google Maps	Map tiles, places, geocode results	Coordinates and address queries
EtelAI agent	Quote, book, status and cancel requests (service key)	Quote, ride status
Telegram / Web Push	-	Operational alerts and rider/driver notifications

6.2 Processes

1. **1.0 Authenticate and authorize.** The user submits phone and password (or an EtelID SSO token). Login is rate-limited (login_attempts, keyed by phone). On success the server issues an HMAC-SHA256 bearer token embedding user id, issue time and the user's current token_version. Every later request passes through boot.php, which verifies the signature (hash_equals), the 7-day lifetime and that token_version still matches D1, so logout or password change revokes all tokens at once. Admin actions additionally check D7 (role_permissions) for the required capability.
2. **2.0 Driver onboarding and compliance.** The driver uploads selfie, licence photo and vehicle details through upload.php (content-sniffed MIME whitelist, size cap, server-generated names). The identity is cross-checked with EtelID; mismatches are flagged for admin review. Admin approval sets ride_drivers.onboarded = 1 in D2. Later vehicle changes create a pending request and only take effect when an admin approves.
3. **3.0 Fare estimation and booking.** The passenger sends pickup, drop-off and vehicle type. ride_fare.php reads the per-vehicle rules from D4, computes the fare server-side (client values are never trusted) and returns a quote. On booking, a ride row is written to D3 with status searching.
4. **4.0 Dispatch and ride lifecycle.** hexgrid.php finds nearby online drivers. A driver must pass the onboarding gate (D2) before accepting. The ride moves searching, accepted, in_progress, completed or cancelled in D3; every transition checks that the caller owns the ride and that the previous status is valid, so a duplicate or replayed action is rejected. Stuck rides are closed by the sweeper and cron script.
5. **5.0 Live tracking, chat and SOS.** The driver app posts GPS points, stored as ride_path_points in D3. SSE endpoints stream status and position to the two ride participants only. Chat messages, issues and SOS events are stored in D5; SOS and operational alerts are forwarded to staff through Telegram and Web Push.

6. **6.0 Payments, earnings and top-up.** On completion the fare is posted to the driver's earnings. For an Etel Money top-up the driver supplies a tx_ref, which is verified server-to-server against EtelPay before a pending row is created in D6; the balance moves only after a second, independent admin approval. Cash-outs are recorded in D6.
7. **7.0 Administration and pricing.** Capability-scoped admin actions edit fare rules (D4), resolve issues (D5), approve drivers and vehicle changes (D2) and approve top-ups (D6). Release channels control which app bundle version users receive.

6.3 Typical end-to-end flow (ride request)

1. Passenger app sends ride_estimate_fare to boot.php with its bearer token. boot.php authenticates, whitelists the action and dispatches to RideController.
2. RideController reads D4, computes the quote and returns it. The passenger confirms and book_ride writes a searching ride to D3.
3. Nearby drivers receive the request. A driver calls driver_accept_ride; the onboarding gate and an atomic status update ensure only one driver wins.
4. driver_arrived, driver_start_ride and driver_update_location advance the ride and record the path while SSE keeps the passenger updated.
5. driver_complete_ride finalises the fare breakdown, stores it on the ride, updates earnings (D6) and prompts the passenger to rate.

6.4 Data in transit and at rest

- All client-server and server-server traffic uses HTTPS; bearer tokens travel in the Authorization header only.
- Passwords are stored hashed; tokens are not stored server-side, only the per-user token_version.
- All SQL is parameterised; dynamic identifiers come from fixed whitelists.
- Uploaded documents are stored under server-generated names; runtime deploy backups are blocked from web access (Require all denied).
- Service-to-service calls use a shared key compared with hash_equals(), with a query-parameter fallback for hosts that strip custom headers.

7. Module Design (Ride-relevant)

7.1 AuthController — Identity & Session

Handles register, login, etelid_login, admin_etelid_login, change_password. Shared unchanged with Etel Delivery. Implements a custom bearer-token scheme (not PHP sessions, not JWT): gen_token()/verify_token() in boot.php produce an HMAC-SHA256-signed token embedding user_id, issued_at, and a token_version counter, giving immediate server-side revocation on logout or password change without a token blacklist table.

7.2 RideController — Ride-Hailing Lifecycle

The largest Ride-relevant module (65 actions, ~210KB). Covers fare estimation and locking, nearby-driver discovery (hex-grid matching), booking, the full driver-side state machine (accept → arrived → start → complete), cancellation, rating, in-ride chat, live GPS path tracking, SOS, favorite routes, scheduled rides, driver earnings/cashout/Etel Money top-up, driver compliance documents, and driver vehicle-change requests. Design note: rides_require_onboarded() is a single shared gate function applied to every driver-privileged action, added specifically so that onboarding-completeness is enforced in exactly one place rather than duplicated per action.

7.3 AdminController — Ride-Relevant Subset

Administration is capability-scoped: every action is gated through require_admin($pdo, 'capability'), checking both the admin role and a specific capability string against the role_permissions table. Ride-relevant capabilities configure ride fare rules and pricing, and resolve ride/driver issues. The same controller also serves Delivery-only administration (order/service-giver/zone management) — those actions are out of scope for this document, but are noted here for transparency since they live in the same file and share the same capability-checking mechanism.

7.4 EqubController — Rotating Savings

19 actions implementing a traditional Ethiopian Equb (rotating savings group) as a benefit available to drivers (and, out of scope here, couriers). Group create/join/leave/start, package selection, contribution tracking, payout rounds, plus admin-side participant/access management.

7.5 AgentController — EtelAI Integration (Ride actions)

agent_ride_quote, agent_book_ride, agent_ride_status, agent_cancel_ride. Every action requires a shared service key (constant-time hash_equals() check against ETELID_SERVICE_KEY); member-specific actions additionally resolve a real etel_sso_token to a specific local user. This controller also exposes Delivery-order actions (agent_order_quote, agent_place_order, etc.), out of scope here.

7.6 NfcController — EtelID NFC Bridge (Ride action)

nfc_start_ride, gated only by the shared service key with no independent role check — a deliberate design choice trusting that EtelID has already verified the scanning party's role before calling in, and independently re-verifying the resolved driver's onboarding status before allowing the ride to start. This controller also exposes courier pickup/delivery confirmation actions for Delivery, out of scope here.

8. Data Architecture (Ride-relevant tables)

Server-Sent Events (SSE) endpoints (sse_ride_status, sse_driver_stream) push live ride status/driver location updates to connected clients, avoiding client-side polling.

Entity	Key relationships	Notes
users	1—1 ride_drivers (via user_id)	Shared identity table for all roles; role column distinguishes passenger/driver/admin. Also used by Delivery (customer/courier/service_giver roles), out of scope here.
rides	N—1 users (passenger_id), N—1 users (driver_id)	Core ride-hailing entity; status ENUM searching/accepted/in_progress/completed/cancelled
ride_drivers	1—1 users	Driver profile: vehicle, identity verification, onboarded flag, earnings
ride_path_points	N—1 rides	GPS breadcrumb trail
ride_fare_rules	referenced by vehicle_type	Per-vehicle-type fare configuration (base fare, per-km rate, wait rate, min fare)
driver_compliance_requests / driver_vehicle_change_requests	N—1 users (driver)	Admin-approved change workflows
driver_payouts / driver_etel_money_topups	N—1 users (driver)	Cash-out and wallet top-up records
ride_messages / ride_typing	N—1 rides	In-ride chat
equb_groups / equb_members / equb_contributions	N—N users via equb_members	Rotating savings (drivers and couriers)
role_permissions	referenced by role_name string, not FK	Capability list per admin role/staff_role

9. Interface Design

9.1 Internal interfaces (client to server)

Server-Sent Events (SSE) endpoints (sse_ride_status, sse_driver_stream) push live ride status/driver location updates to connected clients, avoiding client-side polling.

9.2 External interfaces

System	Direction	Purpose	Auth mechanism
EtelID	Inbound + Outbound	SSO login, driver identity cross-check, NFC-tap ride-start	Shared service key (both directions)
EtelPay	Outbound	Verify driver Etel Money top-up transaction references	etel_pay_verify_tx() server-to-server call

System	Direction	Purpose	Auth mechanism
Telegram Bot API	Outbound	Operational alerting	Bot token
Web Push (VAPID)	Outbound	Browser push notifications	VAPID key pair

9.3 Real-time interfaces

Server-Sent Events (SSE) endpoints (sse_ride_status, sse_driver_stream) push live ride status/driver location updates to connected clients, avoiding client-side polling.

10. Deployment Architecture

10.1 Hosting topology

sw.js implements cache-first for static assets, network-first-with-5-minute-cache for API calls, and a dedicated offline fallback page for navigation. CACHE_NAME must be bumped on every deploy that changes cached files, or clients will continue serving stale bundles despite a successful server-side deploy.

10.2 Release and rollout model

This SDD's modules (Section 2) map 1:1 to the Functional Requirements groups in the SRS and to the controller files cited throughout. The full per-action API contract detail for Ride is maintained in the companion Technical Specifications (TDD/LLD) document.

10.3 Offline / PWA behavior

sw.js implements cache-first for static assets, network-first-with-5-minute-cache for API calls, and a dedicated offline fallback page for navigation. CACHE_NAME must be bumped on every deploy that changes cached files, or clients will continue serving stale bundles despite a successful server-side deploy.

11. Traceability

This SDD's modules (Section 2) map 1:1 to the Functional Requirements groups in the SRS and to the controller files cited throughout. The full per-action API contract detail for Ride is maintained in the companion Technical Specifications (TDD/LLD) document.